Aiqon AI Internship – Take-Home Assignment-5

Project: RouteWise- An Explainable Shipment Route Optimization System

Submitted by: Chittari Suma Priya Varma
GitHub Repository: https://github.com/sumapriya2420030337/routewise-shipment-optimizer
Technology Stack: React,TypeScript, FastAPI,Python

Abstract

RouteWise is a decision-support prototype for selecting shipment routes under cost, delivery-time, and operational-risk constraints. It evaluates synthetic route candidates, rejects options that violate hard limits or supported transport-mode rules, and ranks eligible routes using configurable cost, time, and risk weights. The application combines a React and TypeScript frontend with a Python FastAPI backend. Its interface includes shipment exploration, explainable route comparisons, an endpoint map, sensitivity analysis, hypothetical disruption simulation, and evaluation against baseline strategies. The system is designed to make trade-offs and rejection reasons visible to a logistics operator. Because its data and metrics are synthetic, its results demonstrate software behaviour and decision logic rather than validated real-world logistics performance.

1. Problem, Aim and Objectives

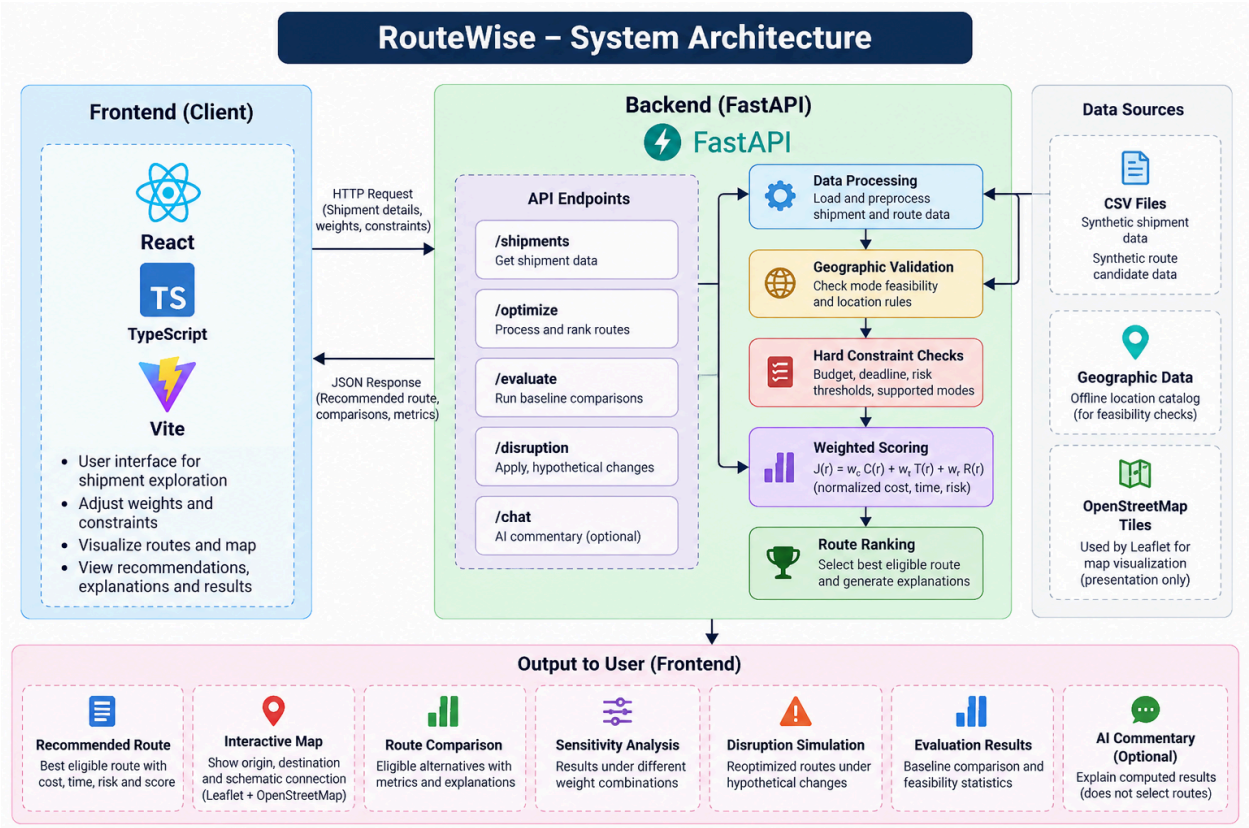
Route selection is a multi-objective decision problem: the cheapest option may be too slow or risky, while the fastest option may exceed a budget. A useful decision-support tool must therefore consider several metrics together, enforce operational limits, and explain why a route is selected or rejected.

Aim. To build a transparent, constraint-aware shipment route optimizer that compares alternatives and explains the trade-offs behind its recommendation.

Objectives. Evaluate multiple candidates per shipment; enforce budget, deadline, risk, and geographic feasibility rules; rank eligible routes according to user priorities; show score contributions and rejection reasons; and examine recommendation behaviour through sensitivity analysis, disruption scenarios, and baseline evaluation.

2. System Architecture

The React/TypeScript interface sends requests to a FastAPI backend. Backend services process shipment and candidate data, validate geographic and hard-constraint feasibility, score eligible routes, and return the recommendation with supporting metrics. CSV files provide the synthetic evaluation dataset. Leaflet and OpenStreetMap provide an endpoint overview in the interface. Optional Groq-powered commentary can explain computed results, but it does not select routes or modify optimizer decisions.



3. Optimization Methodology

For each shipment, RouteWise evaluates candidate routes using cost, transit time, delay probability, congestion risk, geopolitical risk, and transport mode. Hard constraints are applied before weighted ranking. Candidates that exceed the active budget, deadline, or risk thresholds are excluded. Geographic checks also reject unsupported direct modes, such as road or rail across an ocean when explicit transfer segments are absent. If no candidate passes, the system reports that no feasible route is available instead of recommending an invalid option.

$$J(r) = wc\ C(r) + wt\ T(r) + wr\ R(r)$$

Here, C(r), T(r), and R(r) represent the implementation's normalized cost, time, and risk components; the non-negative weights sum to one. The eligible route with the lowest objective value is preferred. Constraint checks determine eligibility; the score only ranks candidates that pass those checks.

4. Application Modules

Module	Purpose
Shipment Explorer	Select and inspect shipments, candidate routes, limits, and feasibility status.
Decision Lab	Adjust cost, time, and risk weights and override constraint thresholds.
Explainable ranking	Show the selected route, objective contributions, and reasons alternatives were rejected.
Route map	Display origin/destination markers and a schematic endpoint connection for an eligible candidate.
Sensitivity analysis	Check whether recommendations change under different weight combinations.
Disruption Lab	Apply hypothetical metric shocks and compare reoptimized outcomes.
Evaluation	Compare baseline strategies and summarize feasibility, cost, time, risk, and objective metrics.
Optional AI commentary	Generate explanatory text from computed results; it does not control route selection.

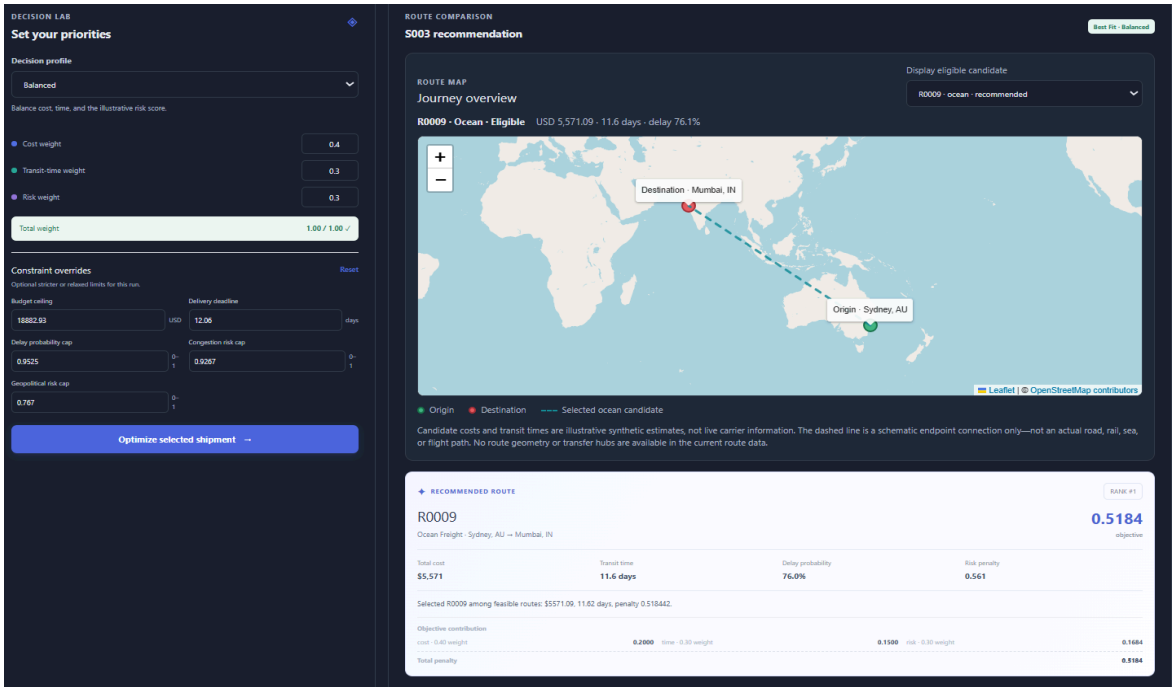
5. Implementation

- **Frontend**: React, TypeScript, and Vite.
- **Backend**: Python and FastAPI.
- **Data**: Synthetic shipment and route data.
- **Route Optimization**: Route-processing and optimization services evaluate candidate routes and rank eligible options.
- **Interactive Map**: Leaflet with OpenStreetMap tiles.
- **Map Visualization:** Displays origin and destination endpoints with a schematic connection.
- **Limitation**: Does not display verified road, rail, sea, or flight paths.

6. Evaluation and Results

The current synthetic dataset contains 30 shipments and 120 candidate routes. In the latest dashboard state, 11 shipments have at least one feasible route and 19 have no feasible option, giving a shipment feasibility rate of 36.7%. Feasibility depends on the active budget, delivery deadline, risk thresholds, and geographic rules.

Selected S003 Sydney, AU→Mumbai,IN



The dashboard currently reports identical aggregate results for the weighted, cheapest, fastest, and reliability-oriented baselines. Therefore, these results do not establish that one strategy outperforms the others.

7. Testing and Verification

The latest development log reports 61 passing backend tests and 20 passing frontend tests across three test files. Frontend linting and the production build also passed. Focused frontend checks cover eligible-candidate selection, unknown coordinates, and map-tile failure handling.

Dataset generation with the same seed produced matching CSV hashes in two runs. Rerun the checks after any further code changes and replace these figures if the results differ.



8. Limitations and Future Work

Synthetic Data: Route metrics are synthetic and do not represent live carrier quotes, calibrated predictions, or proven real-world optimality.

Geographic Feasibility: The offline geographic catalog performs conservative city-level checks rather than verifying specific transport corridors.

Map Limitations: The map displays endpoints and a schematic connection; actual route geometry, multimodal legs, and transfer hubs are not modeled.

Baseline Evaluation: Baseline averages are currently identical, limiting conclusions about which strategy performs best.

Future Improvements:

- Integrate historical shipment or carrier data.
- Implement corridor-aware routing.
- Model explicit multimodal transfer segments and hubs.
- Add persistent shipment storage.
- Validate recommendations against observed shipment outcomes.

9. Conclusion

RouteWise demonstrates a transparent approach to shipment route decision support by separating feasibility checks from weighted ranking and exposing the reasons behind its recommendations. Its dashboard, map, sensitivity tools, disruption simulation, and evaluation views make trade-offs inspectable. The current results support functional validation on synthetic data; real-world performance would require richer route data and external validation.